

1. Jantar dos Filósofos

O problema do Jantar dos Filósofos ilustra disputas por recursos compartilhados. No protocolo ingênuo, cada filósofo pensa, passa para o estado “com fome” e tenta adquirir primeiro o garfo da esquerda e depois o da direita. Quando todos os filósofos fazem isso simultaneamente, cada um acaba segurando um garfo e aguardando o outro, criando uma espera circular. Nenhum filósofo consegue prosseguir, caracterizando um impasse.

Para eliminar esse impasse, foi adotada a solução do garçom (árbitro). Antes de tentar adquirir os garfos, cada filósofo solicita permissão ao garçom. O garçom só permite que filósofos capazes de pegar ambos os garfos prossigam. Com isso, evita-se que vários filósofos adquiram parcialmente os recursos. Como o garçom controla a entrada e garante acesso exclusivo, a condição de espera circular desaparece e o deadlock não pode ocorrer. Essa estratégia quebra uma das condições de Coffman e garante progresso contínuo.

2. Condição de Corrida e Semáforo

A condição de corrida foi demonstrada executando múltiplas threads incrementando um contador compartilhado (`count++`) sem sincronização. A operação de incremento não é atômica; portanto, diferentes threads leem e escrevem o valor simultaneamente, resultando em perda de incrementos — mesmo sem apresentar erros na execução.

Os resultados obtidos foram:

- Sem semáforo:
Esperado = 2.000.000
Obtido = 916.410
Tempo = 0,026s
O valor abaixo do esperado evidencia claramente a condição de corrida.
- Com semáforo:
Esperado = 2.000.000
Obtido = 2.000.000
Tempo = 6,447s
O semáforo binário (`Semaphore(1, true)`) garante exclusão mútua, serializando o acesso ao contador.

A correção funciona porque o semáforo estabelece a relação de happens-before: o `release()` de uma thread garante que todas as suas atualizações na seção crítica ficam visíveis para a thread que executa o próximo `acquire()`. Isso evita leituras desatualizadas e perda de incrementos. Embora o throughput diminua devido à serialização, o resultado passa a ser correto e determinístico.

3. Deadlock com Locks A e B

O terceiro cenário reproduz um deadlock clássico: duas threads e dois locks adquiridos em ordens opostas. A Thread 1 adquire LOCK_A e espera por LOCK_B, enquanto a Thread 2 adquire LOCK_B e espera por LOCK_A. Nenhuma das duas libera o recurso que possui, gerando o impasse.

Nesse caso, as quatro condições de Coffman estão presentes:

- Exclusão mútua: cada lock só pode ser segurado por uma thread.
- Manter-e-esperar: cada thread segura um lock enquanto espera o outro.
- Não-preempção: locks não podem ser retirados de uma thread.
- Espera circular: T1 aguarda B enquanto segura A; T2 aguarda A enquanto segura B.

A solução aplicada foi a hierarquia global de recursos, impondo a ordem fixa de aquisição LOCK_A → LOCK_B para todas as threads. Com essa alteração, a espera circular deixa de existir e o deadlock se torna impossível. Ambas as threads passam a adquirir os locks na mesma ordem e conseguem completar a execução sem travamentos.

Conclusão

Os três experimentos demonstram como erros de sincronização podem gerar impasses, perda de integridade e travamentos completos no sistema. O Jantar dos Filósofos mostrou como arbitrar o acesso resolve o impasse. A condição de corrida evidenciou que acesso concorrente a dados compartilhados sem sincronização leva a inconsistências silenciosas, solucionadas com semáforos e garantias de visibilidade entre threads. O deadlock com locks revelou a importância de impor uma ordem global de recursos para quebrar ciclos perigosos.

Cada solução apresentada segue diretamente as técnicas formais discutidas em aula e elimina de maneira clara o comportamento incorreto observado nos códigos originais.